Sanaverkko

Installation Mode — Implementation Documentation

Project Documentation

March 2026

Contents

1 Overview **2**

2 Audience Potentiometer Strip **3**
- 2.1 Concept . . . 3
- 2.2 Parameter table . . . 3
- 2.3 Audience Parameters window . . . 3
- 2.4 Potentiometer strip rendering . . . 3
  - 2.4.1 Tooltip . . . 4
- 2.5 Knob interaction . . . 4

3 Seed Sentence Bar **5**
- 3.1 Concept . . . 5
- 3.2 Workflow . . . 5
- 3.3 Why a wx overlay instead of pygame keyboard input . . . 5
- 3.4 Overlay destruction . . . 5
- 3.5 Committed sentence display . . . 6
- 3.6 Chip display . . . 6
- 3.7 Simulation pause during input . . . 6

4 Preset Integration **7**
- 4.1 Saving . . . 7
- 4.2 Loading . . . 7

5 Network View Adjustment **8**

6 Technical Reference **9**
- 6.1 Key constants . . . 9
- 6.2 Key methods . . . 9
- 6.3 Data flow . . . 10

Chapter 1

Overview

Installation mode is a set of features added to Sanaverkko that allow the system to be operated in a live-performance or gallery-installation context. In this mode:

- A full-screen pygame canvas displays the word network for an audience.
- A strip of virtual potentiometers at the bottom of the canvas lets a performer or the audience adjust algorithm parameters in real time by dragging the knobs.
- A seed-sentence bar lets a user enter a starting sentence that seeds the word network, bypassing the automatic generation for one cycle.
- All of these controls are saved to and restored from JSON preset files together with the rest of the algorithm parameters.

The implementation lives entirely in `sanaVerkkoCore.py` and was merged into `main` from the branch `controls-for-the-audience` on 26 March 2026.

Chapter 2

Audience Potentiometer Strip

2.1 Concept

Any of the 128 slots defined in `AUDIENCE_PARAMS` can be *exposed* to the audience. An exposed slot appears as a knob in the potentiometer strip rendered at the bottom of the pygame window. Turning the knob writes the mapped parameter value into `self.params` on every frame, overriding whatever the wxPython control panel shows.

2.2 Parameter table

`AUDIENCE_PARAMS` is a list of 128 tuples:

```
(param_key, lo, hi, label)
```

The first 44 entries cover the primary algorithm parameters (logic, audio, rhythm, compressor/ADSR, visual). The remaining 84 slots wrap around the same list cyclically, giving extra reach without hard-coding redundant entries.

2.3 Audience Parameters window

The wxPython control panel has an **Audience Parameters...** button that opens a separate scrollable window. Each row in that window corresponds to one AUDIENCE_PARAMS slot and contains:

- a label showing the parameter name,
- an **Expose** checkbox to show the knob in the pygame strip,
- an **Invert** checkbox to reverse the knob direction (so fully clockwise maps to the minimum value rather than the maximum).

2.4 Potentiometer strip rendering

The strip occupies a fixed-height band (`_POT_H = 80 px`) at the very bottom of the pygame canvas. Each exposed knob is drawn as:

- a circular arc knob with a 270° sweep (225° at min, −45° at max),
- a fill bar beneath the knob showing the current fraction of the range,
- a 7-character abbreviated label below the bar,

- a small numeric value at the very bottom of the strip.

Scroll arrows appear at the left and right edges when there are more exposed knobs than fit horizontally (slot width = 44 px). The user can drag the strip horizontally or use the mouse wheel to scroll.

2.4.1 Tooltip

When the mouse hovers over a knob, the full parameter name is displayed in a small semi-transparent tooltip panel positioned just above the seed bar. The tooltip surface is cached and only re-rendered when focus moves to a different knob.

2.5 Knob interaction

`Action`	**Effect**
`Click + drag up/down`	Increase/decrease the raw pot value (0–127). Sensitivity: ≈ 0.8 units per pixel.
`Mouse wheel over strip`	Scroll visible knobs left/right.
`Click + drag on empty strip area`	Pan the knob row horizontally.

Chapter 3

Seed Sentence Bar

3.1 Concept

The seed sentence bar sits immediately above the potentiometer strip (`_SEED_H = 72 px`). It allows a user to type one or more words that immediately replace the current network nodes, seeding the generative process with a specific starting point.

3.2 Workflow

1. The user clicks anywhere on the seed bar.
2. A borderless `wx.Frame` overlay appears, positioned pixel- precisely over the seed bar using the SDL2 window position (`pygame._sdl2.video.Window.position`).
3. The overlay contains a `wx.TextCtrl` pre-filled with the current draft (if any). The text field has immediate keyboard focus.
4. The user types words separated by spaces. Only characters present in `gematria_table` (a–z, å, ä, ö) are accepted; all others are silently filtered on commit.
5. Pressing **Enter** commits the sentence; pressing **Esc** cancels. The overlay is then destroyed (not merely hidden — see §3.4).

3.3 Why a wx overlay instead of pygame keyboard input

On macOS, when a wxPython application owns the Cocoa event loop, clicking the SDL/pygame window does not reliably steal keyboard focus from the wx control panel. Characters typed by the user go to the invisible wx text field rather than to pygame's event queue, so no `KEYDOWN` events ever arrive.

Embedding a `wx.TextCtrl` in a borderless overlay frame that floats over the pygame window solves the problem: wx always wins the focus race, so we lean into it. The TextCtrl uses the same input machinery as the parameter panel fields that were already known to work.

3.4 Overlay destruction

The overlay is `Destroy()`ed (not `Hide()`d) when the user commits or cancels. A merely hidden `STAY_ON_TOP` `wx.Frame` remains in the macOS Cocoa window responder chain and double-routes keystrokes to whichever text field is active in the main parameter window, causing each typed character to appear twice (e.g. typing `64` produces `6644`). Destruction removes the window from the Cocoa layer completely.

3.5 Committed sentence display

After a successful commit the sentence is shown as a line of text rendered directly onto the pygame canvas, left-aligned just above the seed bar. This text persists until the next commit and serves as a visible reminder to the performer of the current seed.

3.6 Chip display

The committed words are also shown as coloured *chips* in the seed bar itself. Each chip's hue is derived from the word's gematria value (the sum of the Hebrew-style letter values of its characters), scaled against the maximum gematria in the current draft. Chip text surfaces are cached per word to avoid redundant font rendering. The cache is cleared whenever the draft changes.

3.7 Simulation pause during input

While the seed overlay is open (`_seed_active = True`), the main simulation loop resets `last_process_time` every frame. This means the logic worker does not fire immediately when the overlay closes, preventing a burst computation that would stall the event loop and make the seed bar appear unresponsive to subsequent activations.

`_commit_seed_sentence()` also resets `last_process_time` directly, giving one full `process_interval` of lag-free time after each commit regardless of how the overlay was closed.

Chapter 4

Preset Integration

4.1 Saving

When the user saves a preset via **Save Preset**, three additional arrays are appended to the JSON file:

```
preset_data["audience_values"]   = list(self.pot_values)    # 128 ints 0-127
preset_data["audience_exposed"]  = list(self.pot_exposed)   # 128 bools
preset_data["audience_inverted"] = list(self.pot_inverted)  # 128 bools
```

4.2 Loading

When a preset is loaded, the three arrays are read back with element-wise coercion (missing entries default to safe values). If the Audience Parameters window is currently open, its rows are refreshed immediately so the checkboxes reflect the loaded state.

Chapter 5

Network View Adjustment

Before this feature branch the word-network circle was centred on the full window height, causing nodes to overlap with the pot strip and seed bar.

The `makeWordCircle()` method now computes a reduced centre:

```
pot_h    = self._POT_H if self._pot_exposed_indices() else 0
reserved = pot_h + self._SEED_H
centre_y = (self.size[1] - reserved) / 2
```

This shifts the network upward so it occupies only the area above the audience controls, keeping nodes clearly visible at all zoom levels.

Chapter 6

Technical Reference

6.1 Key constants

Name	Value / meaning
`_POT_H`	80 px — height of the potentiometer strip
`_POT_W`	44 px — width per knob slot
`_POT_R`	14 px — knob circle radius
`_SEED_H`	72 px — height of the seed sentence bar
`_SEED_FONT_SIZE`	19 pt — chip label font size
`AUDIENCE_PARAMS`	Module-level list of 128 `(key, lo, hi, label)` tuples

6.2 Key methods

Method	Responsibility
`_pot_exposed_indices()`	Return sorted list of pot indices where `pot_exposed[i]` is True.
`_pot_value_to_param(idx, raw)`	Map raw 0–127 value to the physical parameter range, applying inversion.
`_apply_pot_values_to_params()`	Called every frame; writes all exposed pot values into `self.params`.
`_draw_pot_strip(screen)`	Render all visible knobs, scroll arrows, and the hover tooltip.
`_handle_pot_events(events)`	Process mouse drag/scroll on knobs; update hover index.
`_draw_seed_editor(screen)`	Render the seed bar background, chips, and (when overlay active) a dim overlay tint.
`_handle_seed_events(events)`	Detect click on seed bar; call `wx.CallAfter(_show_seed_overlay)`.
`_show_seed_overlay()`	Create and display the borderless wx input frame.
`_hide_seed_overlay(cancel, text)`	Destroy the overlay; on commit, parse text and call `_commit_seed_sentence()`.
`_commit_seed_sentence()`	Replace network nodes with seed words; reset audio signature; delay next logic tick.

Method	Responsibility
`_seed_bar_rect()`	Return `(x, y, w, h)` of the seed bar in pygame window coordinates.
`_seed_overlay_screen_rect()`	Translate seed bar rect to absolute screen coordinates using SDL2 window position.
`_seed_recompute_gem_max()`	Recompute the maximum gematria value across the current draft for chip colour scaling.
`OnOpenAudienceParams(event)`	Open the Audience Parameters scrollable wxPython window.
`OnSavePreset(event)`	Save all params plus audience arrays to JSON.
`OnLoadPreset(event)`	Restore params and audience arrays from JSON.

6.3 Data flow

1. **Every simulation frame:**
   (a) `_handle_pot_events` processes mouse events and updates `pot_values[i]`.
   (b) `_handle_seed_events` checks for a click on the seed bar and posts `_show_seed_overlay` if needed.
   (c) `_draw_seed_editor` and `_draw_pot_strip` render the bottom UI bands.
2. **On each rate-limited logic tick** (governed by `process_interval`):
   (a) `_apply_pot_values_to_params` writes knob values into `self.params`.
   (b) The logic worker reads `self.params` for all simulation decisions.
3. **On seed commit:**
   (a) `_commit_seed_sentence` replaces `self.words` and `self.referenceWords`.
   (b) Audio signature is invalidated so `updateAudio()` re-synthesises on the next tick without interrupting the currently playing audio.